E2E Deployment

This deployment keeps the CloudGuard portal/GRC layer separate from heavy scanner execution.

Target Layout

- **Portal VM:** dashboard, GRC module, FastAPI API, PostgreSQL, scheduler in enqueue-only mode, and optional MinIO object storage. Public HTTPS/routing should be handled by Dokploy/Traefik or another external reverse proxy.
- **Worker VM:** local scan workers that process queued jobs directly against the shared database.
- **Sandbox Worker:** provisions temporary vulnerable IaC/cloud/Kubernetes demo labs, stores validation proof, and destroys resources after scan completion or TTL expiry.
- **Connector VM/Service:** file/API connectors that normalize evidence and push it to `/api/evidence`.

The portal does not execute scanner or sandbox workloads in this model. It queues jobs and stores/displays evidence.

This compose file does not bind ports `80` or `443`; that avoids conflicts on a shared VM where Dokploy should own public domain routing and SSL.

By default, the backend binds only to `127.0.0.1:8000`. A reverse proxy or tunnel can route public traffic to that local backend port without taking over `80/443`.

1. Portal VM

Install Docker on the VM, then create an env file:

```
cp deploy/e2e/env.portal.example .env.e2e.portal
nano .env.e2e.portal
```

Start manually:

```
docker compose -f deploy/e2e/portal-compose.yml --env-file .env.e2e.portal up -d --build
```

Or deploy from your laptop:

```
./deploy/e2e/deploy.sh portal ubuntu@PORTAL_VM_IP .env.e2e.portal
```

2. Worker / Scanner VM

Create a worker env file:

```
cp deploy/e2e/env.worker.example .env.e2e.worker
nano .env.e2e.worker
```

Important:

- `DATABASE_URL` must point to the shared PostgreSQL database.
- `EVIDENCE_CONNECTOR_ID` must identify an entry in the portal's `CONNECTOR_CREDENTIALS_JSON`, and `EVIDENCE_CONNECTOR_TOKEN` must match that entry's token.
- Object storage settings should point to shared object storage. For separate VMs, prefer E2E object storage or MinIO exposed only on a private network.
- Keep `SANDBOX_AWS_DEPLOY`, `SANDBOX_GCP_DEPLOY`, and `SANDBOX_KUBERNETES_DEPLOY` disabled until isolated sandbox accounts/projects/clusters are ready.

Start manually:

```
docker compose -f deploy/e2e/worker-compose.yml --env-file .env.e2e.worker up -d --build
```

Or deploy from your laptop:

```
./deploy/e2e/deploy.sh worker ubuntu@WORKER_VM_IP .env.e2e.worker
```

Keep Scanner Workers Online

Worker services use `restart: always` and continuously write heartbeats. The portal shows scanner availability at `/operations` and exposes the same data through `/api/workers/status`.

After deployment or VM reboot, verify:

```
docker compose -f deploy/e2e/portal-compose.yml --env-file .env.e2e.portal ps
docker compose -f deploy/e2e/worker-compose.yml --env-file .env.e2e.worker ps
curl -fsS http://127.0.0.1:8000/api/workers/status
```

If scanner availability is offline, restart the worker VM stack:

```
docker compose -f deploy/e2e/worker-compose.yml --env-file .env.e2e.worker up -d --build
```

3. Connector Contract

Connectors submit evidence to:

```
POST /api/evidence
Header: Authorization: Bearer <tenant-specific-connector-token>
Header: X-Connector-ID: <connector-id>
```

Payload:

```
{
  "evidence_id": "optional-id",
  "job_id": "optional-job-id",
  "control_id": "A.1.3.7",
  "control_name": "Enable access controls",
  "source_system": "trivy",
  "scanner_type": "container",
  "artifact_type": "json",
```

```
  "filename": "trivy-result.json",
  "content_type": "application/json",
  "payload": {
    "findings": []
  },
  "metadata": {
    "connector": "trivy-file"
  }
}
```

The bundled `evidence-connector` service watches `/app/connectors/inbox`. Any JSON file dropped there is normalized and posted to the portal.

4. Security Notes

- Do not expose PostgreSQL publicly. Use E2E private networking, firewall allowlists, or managed PostgreSQL.
- Do not expose MinIO publicly unless it is behind strict firewall rules.
- Give every connector a separate secret and tenant-scoped entry in `CONNECTOR_CREDENTIALS_JSON`; rotate a connector's token independently if it is exposed.
- Keep scanner workers separate from the portal VM for production.